Assignment - 1

Page No.:
Date:

Q1 0-1 Knapsack Problem :

Approach ,

→ To solve this problem ~~we have to~~ the first approach was to create a function solve ~~with the parameters give~~ and use recursion. We find out two values named take & skip which calculates the profit if current value is taken or not respectively.

→ Since using only recursion ~~the~~ all test cases could not be passed, dp was used with two dimensions to optimize the program.

→ Among the two values take & skip the function returns the value with max profit.

Pseudocode ;

```
Class Solution {
    public int Knapsack (int W, int val[],
                         int wt[]) {

        int [][] dp = new int[W+1][val.
                                  [val.length+1];
        return solve (W, val, wt, 0, dp);
    }
```

Page No.:
Date:

```
public int solve (int capacity, int[] val,
                  int wt, int i, int[][] dp){

    if (i==val.length) return 0;
    if (dp[capacity][i]!=0)return dp[capacity][i];
    int take = 0;
    int skip = 0;
    if (wt[i]<=capacity){
        take = val[i]+solve (capacity-
                             wt[i], val, wt
                             ,i+1,dp);
    }
    skip = solve (capacity ,val, wt,
                  i+1, dp);
    return dp [capacity][i]
              = Math.max (take, skip);

  }
}
```